IMPLEMENTATION PLAN — V1.0

WorkForce Hub

Employee Management System · Django REST Framework Backend

Framework Django 5.x + DRF 3.x | **Language** Python 3.12+ | **Database** SQLite / PostgreSQL | **Auth** JWT (SimpleJWT) | **API Docs** drf-spectacular

📋 TABLE OF CONTENTS

1. Project Overview & Tech Stack
2. Setup Instructions
3. Project Structure
4. Database Models (ERD)
5. Roles & Responsibilities
6. Roles & Permissions Matrix
7. API Endpoints
8. Implementation Roadmap
9. Key Settings
10. Testing
11. Future Enhancements

1. Project Overview & Tech Stack

ITEM	DETAIL
Project Name	WorkForce Hub
Framework	Django 5.x + Django REST Framework 3.x
Language	Python 3.12+
Database	SQLite (development) / PostgreSQL (production)
Authentication	JWT via `djangorestframework-simplejwt`
API Testing	Thunder Client / Postman
API Docs	`drf-spectacular` — Swagger UI & ReDoc

PACKAGE	PURPOSE
`django`	Core web framework
`djangorestframework`	REST API toolkit
`djangorestframework-simplejwt`	JWT auth — access + refresh tokens
`drf-spectacular`	Auto-generated OpenAPI 3.0 schema & Swagger UI
`django-filter`	Queryset filtering on list endpoints
`django-cors-headers`	CORS handling for external API clients
`python-decouple`	Environment variable management
`pillow`	Image handling (employee profile photos)
`pytest` / `pytest-django`	Automated testing

2. Setup Instructions

Step 1 — Clone & Create Virtual Environment

```
# Clone the repository
git clone <your-repo-url>
cd WorkForce_Hub

# Create virtual environment
python -m venv venv

# Activate — Windows:
venv\Scripts\activate
# Activate — Mac/Linux:
source venv/bin/activate
```

Step 2 — Install Dependencies

```
pip install -r requirements.txt
```

Step 3 — Configure .env File

```
# Create .env file
cp .env.example .env

# Contents of .env:
SECRET_KEY=your-secret-key-here
DEBUG=True
DATABASE_URL=sqlite:///db.sqlite3
ALLOWED_HOSTS=127.0.0.1,localhost
```

Step 4 — Database Setup

```
python manage.py makemigrations
python manage.py migrate
python manage.py createsuperuser   # Create Admin account
```

Step 5 — Run Development Server

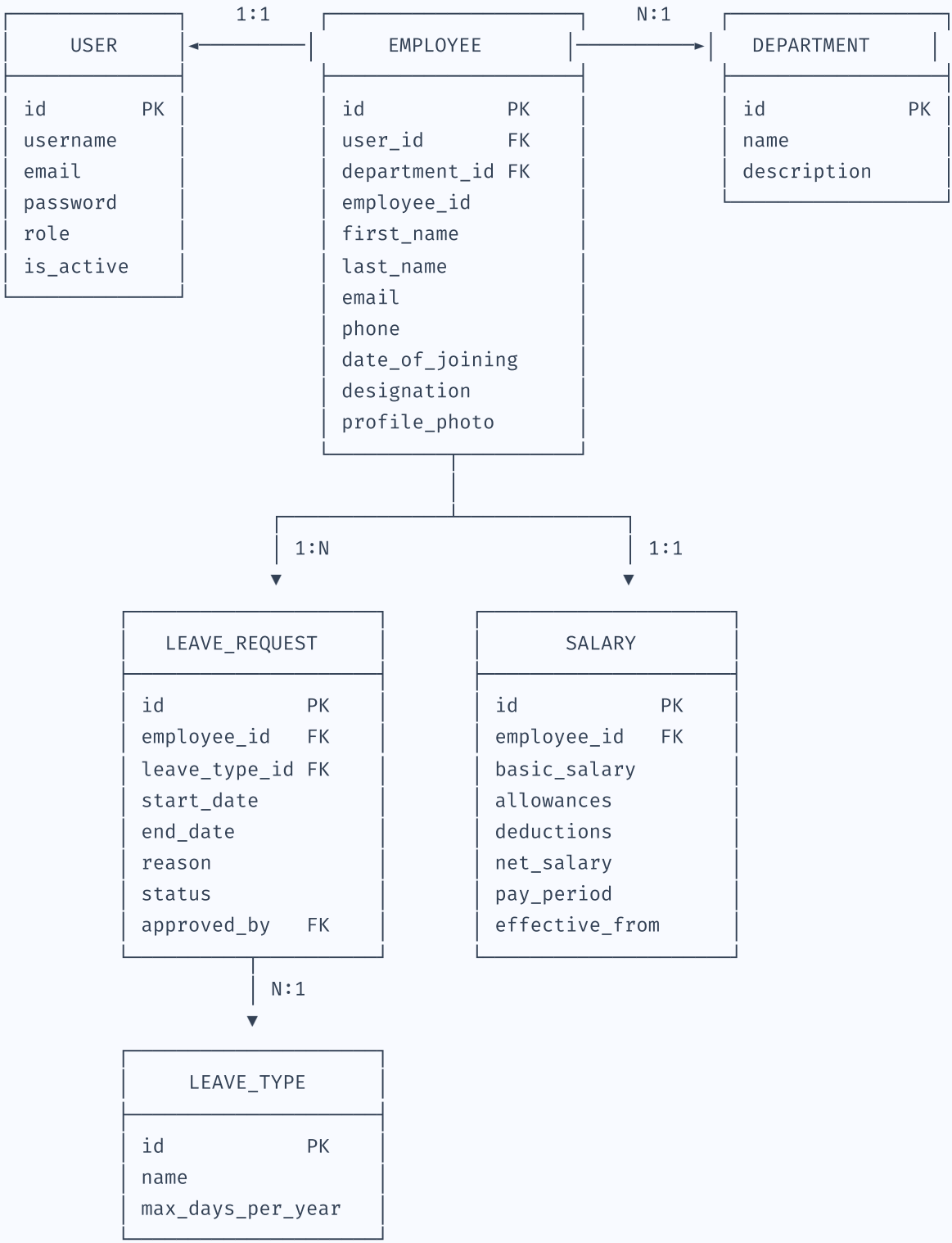
```
python manage.py runserver

# API →     http://127.0.0.1:8000/api/
# Swagger → http://127.0.0.1:8000/api/docs/
# Admin →   http://127.0.0.1:8000/admin/
```

3. Project Structure

```
WorkForce_Hub/                      ← Project root
│
├── manage.py
├── requirements.txt
├── .env                            ← Secret key, DB creds, DEBUG flag
│
├── config/                         ← Project settings package
│   ├── settings.py                 ← Main settings
│   ├── urls.py                     ← Root URLs (Swagger included)
│   └── wsgi.py / asgi.py
│
├── accounts/                       ← Authentication & user management
│   ├── models.py                   ← Custom User model (role field)
│   ├── serializers.py
│   ├── views.py
│   ├── permissions.py              ← IsAdmin, IsHR, IsManager …
│   └── urls.py
│
├── employees/                      ← Core employee CRUD
│   ├── models.py                   ← Employee, Department models
│   ├── serializers.py
│   ├── views.py
│   ├── filters.py
│   └── urls.py
│
├── leaves/                         ← Leave management
│   ├── models.py                   ← LeaveType, LeaveRequest
│   ├── serializers.py
│   ├── views.py
│   └── urls.py
│
└── payroll/                        ← Salary & payroll
    ├── models.py                   ← Salary model
    ├── serializers.py
    ├── views.py
    └── urls.py
```

4. Database Models — ERD



SOURCE	TARGET	TYPE	DESCRIPTION
`USER`	`EMPLOYEE`	1 : 1	One user → one employee profile
`DEPARTMENT`	`EMPLOYEE`	1 : N	One department → many employees
`EMPLOYEE`	`LEAVE_REQUEST`	1 : N	One employee → many leave requests
`LEAVE_TYPE`	`LEAVE_REQUEST`	1 : N	One leave type → many requests

SOURCE	TARGET	TYPE	DESCRIPTION
EMPLOYEE	SALARY	1 : 1	Each employee → one salary record

5. Roles & Responsibilities

👑 Admin

Full system control — Highest Authority

- → Create, update, delete any user or employee
- → Assign roles (HR, Manager, Employee) to users
- → Create, edit, delete departments
- → Set, view, and update any employee's salary
- → Approve or reject any leave — system-wide
- → Manage leave types, pay periods, global settings
- → Access all employee, salary, and leave reports

👩‍💼 HR (Human Resources)

People operations — all departments

- → Add new employees to the system (onboarding)
- → View and update employee profiles across all departments
- → View all leave requests; assist in approvals
- → View salary data (read-only, no edit access)
- → View department info — cannot create/delete
- → Generate HR reports — headcount, leaves
- → Manage leave types (Sick, Casual, Earned)

👨‍💻 Manager

Team-level authority — own department only

- → View employees within their own department
- → Update profiles of employees in their department
- → Approve or reject leave requests from own team
- → View salary records of own department employees
- → Track performance within their team

👤 Employee

Self-service access only

- → View and update their own profile
- → Apply for leave (Sick, Casual, Earned)
- → View status of their own leave requests
- → View their own salary slip / details
- → Change their own account password

6. Roles & Permissions Matrix

ACTION	ADMIN	HR	MANAGER	EMPLOYEE
Create employee	✅	✅	❌	❌
View all employees	✅	✅	✅ (own dept)	❌
Update any employee	✅	✅	✅ (own dept)	❌
Update own profile	✅	✅	✅	✅
Delete employee	✅	❌	❌	❌
Assign roles	✅	❌	❌	❌
Manage departments	✅	❌	❌	❌
View departments	✅	✅	✅	✅
Approve / reject leave	✅	✅	✅ (own dept)	❌
Apply for leave	✅	✅	✅	✅
View all leave requests	✅	✅	✅ (own dept)	❌
View own leave history	✅	✅	✅	✅
Create leave types	✅	✅	❌	❌
View all salaries	✅	✅ (read-only)	✅ (own dept)	❌
View own salary	✅	✅	✅	✅
Update salary	✅	❌	❌	❌
Access admin panel	✅	❌	❌	❌

Custom Permission Classes: `IsAdmin` · `IsHR` · `IsManager` · `IsEmployee` · `IsAdminOrHR` · `IsManagerOrAbove` · `IsOwnerOrAdmin`

7. API Endpoints

7.1 Authentication `/api/auth/`

METHOD	ENDPOINT	DESCRIPTION	ACCESS
POST	`/auth/register/`	Register new user account	Public
POST	`/auth/login/`	Login — obtain JWT tokens	Public
POST	`/auth/token/refresh/`	Refresh access token	Authenticated
GET	`/auth/profile/`	Get current user's profile	Authenticated
PUT	`/auth/change-password/`	Change own password	Authenticated

7.2 Departments `/api/departments/`

METHOD	ENDPOINT	DESCRIPTION	ACCESS
GET	`/departments/`	List all departments	Authenticated
POST	`/departments/`	Create new department	Admin
GET	`/departments/{id}/`	Retrieve a department	Authenticated
PUT	`/departments/{id}/`	Update department	Admin
DELETE	`/departments/{id}/`	Delete department	Admin

7.3 Employees `/api/employees/`

METHOD	ENDPOINT	DESCRIPTION	ACCESS
GET	`/employees/`	List all employees (filter & search)	Admin HR Manager
POST	`/employees/`	Create / onboard new employee	Admin HR
GET	`/employees/{id}/`	Retrieve employee record	Admin HR Manager* Owner
PUT	`/employees/{id}/`	Update employee profile	Admin HR Manager*
DELETE	`/employees/{id}/`	Delete / offboard employee	Admin
GET	`/employees/me/`	Get own employee profile	Authenticated

* Manager: own department only

7.4 Leave Management `/api/leaves/`

METHOD	ENDPOINT	DESCRIPTION	ACCESS
GET	/leaves/types/	List leave types	Authenticated
POST	/leaves/types/	Create new leave type	Admin HR
GET	/leaves/requests/	List all leave requests	Admin HR Manager*
POST	/leaves/requests/	Apply for leave	Authenticated
GET	/leaves/requests/{id}/	Retrieve leave request	Admin HR Manager* Owner
PATCH	/leaves/requests/{id}/approve/	Approve leave request	Admin HR Manager*
PATCH	/leaves/requests/{id}/reject/	Reject leave request	Admin HR Manager*
GET	/leaves/requests/my/	View own leave history	Authenticated

* Manager: own department only

7.5 Payroll & Salary `/api/payroll/`

METHOD	ENDPOINT	DESCRIPTION	ACCESS
GET	/payroll/	List all salary records	Admin HR (read-only) Manager*
POST	/payroll/	Create salary record	Admin
GET	/payroll/{id}/	Retrieve salary record	Admin HR Manager* Owner
PUT	/payroll/{id}/	Update / revise salary	Admin
GET	/payroll/my/	View own salary slip	Authenticated

* Manager: own department only. HR has read-only access system-wide.

🗺️ 8. Implementation Roadmap

Phase 1 — Project Setup

- [ ] Create virtual environment & install dependencies
- [ ] Initialise Django project (`config`) and create all four apps
- [ ] Configure `settings.py` — installed apps, REST framework, JWT, CORS
- [ ] Set up `.env` file

Phase 2 — Authentication & Users

- [ ] Create custom `User` model with `role` field (`admin` , `hr` , `manager` , `employee`)
- [ ] Build registration & login serializers
- [ ] Add JWT token obtain/refresh views
- [ ] Create profile & change-password endpoints
- [ ] Write custom permission classes (`IsAdmin` , `IsHR` , `IsManager` , `IsOwnerOrAdmin`)

Phase 3 — Employee & Department CRUD

- [ ] Define `Department` and `Employee` models
- [ ] Build serializers with nested department display
- [ ] Create `ModelViewSet` for both with pagination, search, filtering
- [ ] Wire up permission classes per action
- [ ] Add `employees/me/` action for self-profile

Phase 4 — Leave Management

- [ ] Define `LeaveType` and `LeaveRequest` models
- [ ] Build serializers (read-only status for non-admins)
- [ ] Create viewsets + custom actions for approve/reject
- [ ] Add `my/` endpoint for leave history

Phase 5 — Payroll / Salary

- [ ] Define `Salary` model
- [ ] Build serializers (restrict sensitive fields for non-admin)
- [ ] Create viewset with `my/` action for own salary
- [ ] Restrict create/update to Admin only

Phase 6 — API Documentation

- [ ] Integrate `drf-spectacular`
- [ ] Expose `/api/schema/` , `/api/docs/` (Swagger UI), `/api/redoc/`
- [ ] Add `@extend_schema` decorators for custom actions

Phase 7 — Testing & Validation

- [ ] Write `pytest` unit tests per app (models, serializers)
- [ ] Write integration tests (API endpoint tests with `APIClient`)
- [ ] Manual testing via Thunder Client collection
- [ ] Validate role-based access for every endpoint

9. Key Settings

```python
# config/settings.py

REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': (
        'rest_framework_simplejwt.authentication.JWTAuthentication',
    ),
    'DEFAULT_PERMISSION_CLASSES': (
        'rest_framework.permissions.IsAuthenticated',
    ),
    'DEFAULT_PAGINATION_CLASS': 'rest_framework.pagination.PageNumberPagination',
    'PAGE_SIZE': 10,
    'DEFAULT_FILTER_BACKENDS': (
        'django_filters.rest_framework.DjangoFilterBackend',
        'rest_framework.filters.SearchFilter',
        'rest_framework.filters.OrderingFilter',
    ),
    'DEFAULT_SCHEMA_CLASS': 'drf_spectacular.openapi.AutoSchema',
}

SIMPLE_JWT = {
    'ACCESS_TOKEN_LIFETIME': timedelta(minutes=30),
    'REFRESH_TOKEN_LIFETIME': timedelta(days=1),
    'ROTATE_REFRESH_TOKENS': True,
    'BLACKLIST_AFTER_ROTATION': True,
}
```

10. Testing

Automated Tests

```
# Run full test suite
pytest --tb=short -v

# Per-app tests
pytest accounts/tests.py -v
pytest employees/tests.py -v
pytest leaves/tests.py -v
pytest payroll/tests.py -v
```

Manual Testing Flow (Thunder Client / Postman)

#	FLOW	STEPS
1	**Auth**	Register → Login → Copy access token → Use as `Bearer` header
2	**CRUD**	Create department → Create employee → Update → Delete
3	**Role tests**	Login as Employee → hit Admin-only endpoint → Expect `403 Forbidden`
4	**Leave flow**	Apply leave → Login as Manager/HR → Approve → Verify status change
5	**Salary**	Admin creates record → Employee views own salary → HR views all

11. Future Enhancements

FEATURE	NOTES
Attendance tracking	Clock-in / clock-out with timestamps per employee
Export to CSV / PDF	Download salary slips and employee reports
Dashboard & Reports	Summary stats — headcount, leaves pending, payroll total
Search & Filters	Filter employees by department, role, joining date